VERSION 2.0 PROJECT DOCUMENTATION

MetriGo

by **Lanchenba Mutum**

Smart metro booking for the Delhi Metro

MetriGo plans and books trips across all nine Delhi Metro lines. It finds the shortest route with **Dijkstra's algorithm**, lists departures from a timetable shared by the server and the browser, suggests seats that keep a group together (**greedy, then backtracking**), prices each trip by distance and time of day, and takes payment.

Every booking ends in a **QR ticket that opens with no signal**, and a live map shows every simulated train moving. It is a student portfolio project and is not affiliated with DMRC: tickets are not valid for travel and payments run in test mode.



THE NETWORK

Red	29
Yellow	37
Blue	57
Green	24
Violet	34
Pink	46
Magenta	33
Grey	4
Airport Express	7

Stations per line, branches included. Map drawn from the app's own OpenStreetMap coordinates.

244
stations on 9 lines, joined at 29 interchanges

8,505
sampled routes solved, 0 failures, median 0.39 ms each

6,786
timetabled train runs a day, shared by server and live map

$0
monthly running cost on free tiers

AUTHOR
Lanchenba Mutum

SOURCE CODE
github.com/1anc6/Metrigo

STATE DOCUMENTED
20 October 2026

STACK
Node.js · Express 5 · libSQL / Turso · vanilla JS

HOSTING
Vercel (app + API) · Turso (database)

MAP DATA
© OpenStreetMap contributors, ODbL

Contents

Twenty-one stops, from the big picture to the honest assessment.

01	Summary and key numbers	what was built
02	The problem, and what MetriGo does about it	approach
03	Timeline and time spent	v1 to v2
04	System architecture	life of a booking, API
05	The metro network data	stations, edges, lines
06	The algorithms	study this one
07	Benchmark and efficiency	measured numbers
08	Payments and booking integrity	never sell a seat twice
09	Database design	nine tables
10	Frontend: the web app	pages, offline, design
11	Security, privacy and consent	risks and protections
12	Testing and quality	what is checked today
13	Deployment and operations	Vercel, Turso, config
14	Performance and cost	$0 a month
15	How MetriGo compares with other tools	approach, not accuracy
16	Limitations	read before relying on it
17	Version 1 versus version 2	plan versus delivery
18	Complete inventory of everything used	libraries, data, services
19	Future work	next stops
20	Glossary for study	terms defined
21	Licences, credits and publishing notes	before you publish

How to read this document

Chapters 1–4 give the big picture. Chapters 5, 6, 8 and 9 explain the internals and are the ones to study for viva or interview questions; chapter 6 holds every algorithm with its complexity. Chapters 15–17 are the honest assessment.

Where the numbers come from

Every timing and route count was measured by `documentation/benchmark.js` against the real Express app and saved to `benchmark-results.json`. Network facts come from `Backend/data/*.csv` and `metro-sim.js`. Nothing is estimated unless it says so.

01

CHAPTER 01 · RED LINE

Summary and key numbers

MetriGo is a full-stack web app for the Delhi Metro. A rider picks two stations, gets the shortest route across all nine lines, chooses a train and seats, pays, and carries a QR ticket that works offline. It runs from one Node.js codebase locally and on Vercel's free tier.

What was built

- **A web app** in plain HTML, CSS and JavaScript with no framework and no build step: 13 pages, a mobile-first neumorphic design in light and dark, and a desktop sidebar layout. It installs as an app (PWA) and opens saved tickets with no signal.
- **An API** on Express 5 with 23 endpoints for accounts, routing, journeys, seats, payments, bookings, the Metro Card and support. On Vercel the whole app runs as one serverless function.
- **A network model** of 244 stations and 293 track sections from OpenStreetMap, with every line and branch: Red, Yellow, Blue, Green, Violet, Pink, Magenta, Grey and the Airport Express.
- **A timetable simulator** (`metro-sim.js`) that the server and the browser both load, so the train a rider books is the same one moving on the live map.
- **Four classic algorithms** written by hand: Dijkstra with a binary heap for routes, a greedy pass and a pruned backtracking search for seats, and SHA-256 hashing for PNRs and reset codes.
- **A booking engine** that holds seats for 15 minutes during checkout, re-checks them inside a database transaction, settles each payment exactly once, and refunds automatically if the seats went to someone else.
- **Payments** through a gateway adapter: test mode by default, Razorpay Checkout and webhooks when keys are set, plus a Metro Card wallet at 10% off.
- **Ticket exports**: QR code, boarding-pass PDF, the phone's share sheet and a calendar event with a 15-minute reminder.
- **Privacy and legal**: privacy, terms, refund and cookie pages; consent and an age check recorded at sign-up; self-serve account deletion; no trackers.

The guiding principle

The server is the source of truth for anything that costs money, and a ticket must open without a network. Prices are computed on the server and never read from the browser. Seats are re-checked inside the same transaction that writes the booking, and a unique index backs that up. A payment is claimed with an atomic update, so it settles once even if the browser and the webhook arrive together. Confirmed tickets are copied to the phone's IndexedDB so they open in a tunnel.

244
Stations (275 graph nodes, one per line at interchanges), joined at 29 interchanges

9 lines
13 line segments including branches, from OpenStreetMap route relations

397 km
Network length in the model (293 edges; straight-line distance × 1.15)

8,505
Sampled station pairs routed with 0 failures; median request 0.39 ms

6,786
Train runs a day in the shared timetable; 484 trains on the map at 9 AM

160
Seats per train: 4 coaches × 10 rows × 4 seats, the first coach women-only

23
API endpoints, all JSON; signed-in routes answer 401 without a session

~6,450
Lines of JavaScript, HTML and CSS in 23 source files (vendored libraries excluded)

$0
Monthly running cost: Vercel Hobby and Turso free tiers, payments in test mode

Next station **02 · The problem, and what MetriGo does about it**

02

CHAPTER 02 · YELLOW LINE

The problem, and what MetriGo does about it

The problem

The Delhi Metro is one of the world's larger metro networks: nine coloured lines, several branches and dozens of interchanges. Planning a trip means reading a dense map, guessing where to change, and working out the fare. A booking system on top of that has harder problems that a rider never sees: two people must never get the same seat, a payment must never be charged twice or charged without a ticket, and a ticket must still open underground with no signal.

MetriGo started as a Design and Analysis of Algorithms (DAA) project: show classic algorithms doing real work. Version 2 keeps that goal and adds what a real booking product needs around them.

The approach: one job per technique

NEED	HOW METRIGO MEETS IT	HONEST CAVEAT
Find the route	Dijkstra's algorithm with a binary heap over the station graph; changing lines costs a 5 km penalty so routes do not hop lines to save a few hundred metres	Distances are straight-line × 1.15, not surveyed track lengths
Show real departures	A deterministic timetable: trains every 3 minutes at peak and 5 otherwise, 06:00–23:00, each leg scheduled with a 4-minute change	Simulated from averages, not DMRC's timetable or live feeds
Keep a group together	"Best seats": a greedy pass for adjacent seats in one row, then a pruned backtracking search that scores rows, spread and gaps	Real Delhi Metro trains have no reserved seats; this is an allocation exercise
Price fairly	Distance slabs (₹10–₹60), then a time-of-day multiplier from ×0.8 to ×1.4; 10% off with the Metro Card	A model of dynamic pricing, not DMRC's fare table
Take money safely	Server-side amounts, 15-minute seat holds, HMAC-verified payments, one-time settlement, automatic refunds	Test mode by default; live payments need Razorpay keys and rewritten legal pages
Work underground	Service worker caches the whole app; confirmed tickets live in IndexedDB; QR codes render on the phone	Booking and cancelling still need the network
Feel alive	A Leaflet map with every train interpolated between stations and arrival boards for any station	Positions come from the timetable, not GPS

Next station **03 · Timeline and time spent**

03

CHAPTER 03 · BLUE LINE

Timeline and time spent

MetriGo was built twice. Version 1 was uploaded in April 2026 as a DAA course project. Version 2, in October 2026, rebuilt it into a deployable product. The repository holds 25 commits.

Version 1: the course project (29 April 2026)

Uploaded through GitHub's web interface between 21:25 and 22:01 in 17 commits. It had nine HTML pages with a shared `script.js` and `style.css`, data stored in JSON files (`users.json`, `bookings.json`), a 74-station network covering the Blue and Yellow lines, and `MetriGo_README.txt` describing the DAA concepts: Dijkstra, greedy, backtracking, hashing and dynamic pricing. The `node_modules` folder was committed too.

Version 2: the rebuild (15–16 October 2026)

WHEN (IST)	COMMIT	SIZE
15 Oct, 21:05	**Rebuild:** Express backend, neumorphic UI, PWA, SQLite. Committed `node_modules` removed	626 files, +5,633 / −76,290
15 Oct, 21:07	README with setup, configuration, API and architecture	—
15 Oct, 21:15	Prepare for Vercel: Turso database, serverless entry	9 files, +796
15 Oct, 21:32	Vercel: install API dependencies from a root `package.json`	3 files, +30
15 Oct, 21:43	Seamless page transitions and touch sounds	12 files, +293
15 Oct, 21:57	Ticket PDF, share and calendar; styled station picker; QR fix	8 files, +524
16 Oct, 17:05	Legal pages, consent, account deletion and accessibility fixes	35 files, +1,918
16 Oct, 17:19	Every Delhi Metro line (74 → 244 stations); fix "Show ticket" on Home	16 files, +878

Honest estimate of time. Hours were not tracked with a timer, and git shows only when work was saved. The commit windows add up to under two hours (about 0.6 h in April, 0.9 h on 15 October and 0.25 h on 16 October), but the rebuild landed as one large commit, so nearly all of its work happened before it and is invisible to git. If a report needs a number, give your own estimate of design, coding and testing time and cite these windows only as the documented minimum.

What the rebuild changed

AREA	VERSION 1 (APRIL)	VERSION 2 (OCTOBER)
Storage	JSON files rewritten on every change	SQLite through libSQL; Turso in production; transactions; legacy JSON imported once
Network	74 stations, Blue and Yellow lines	244 stations, all 9 lines and branches, from OpenStreetMap
Shortest path	README describes a C module (`dijkstra.c`)	Dijkstra in JavaScript inside the server, with a hand-written binary heap and a transfer penalty

AREA	VERSION 1 (APRIL)	VERSION 2 (OCTOBER)
Timetable	Time slots without real trains	Shared timetable: dated departures, train IDs, multi-leg journeys, live map
Payments	None	Gateway adapter, seat holds, settlement, refunds, Metro Card wallet
Offline	None	Service worker app shell and offline QR tickets
Hosting	Run locally	Vercel serverless function and static files; Turso database
Repository	`node_modules` committed	Dependencies installed by npm; data files and secrets git-ignored

A useful case study: from two lines to nine

The last commit grew the network from 74 to 244 stations. Because routing, the timetable, the live map and the station picker all read the same data (`stations.csv`, `edges.csv` and the `LINES` table), adding seven lines needed no change to any algorithm. The one design decision it forced was **how to model interchanges**: each interchange became one node per line joined by 0 km edges, and Dijkstra charges each change as 5 km. Without that penalty, the bigger network produced routes that changed lines to save a few hundred metres, which no rider would choose.

Next station **04 · System architecture**

04

CHAPTER 04 · GREEN LINE

System architecture

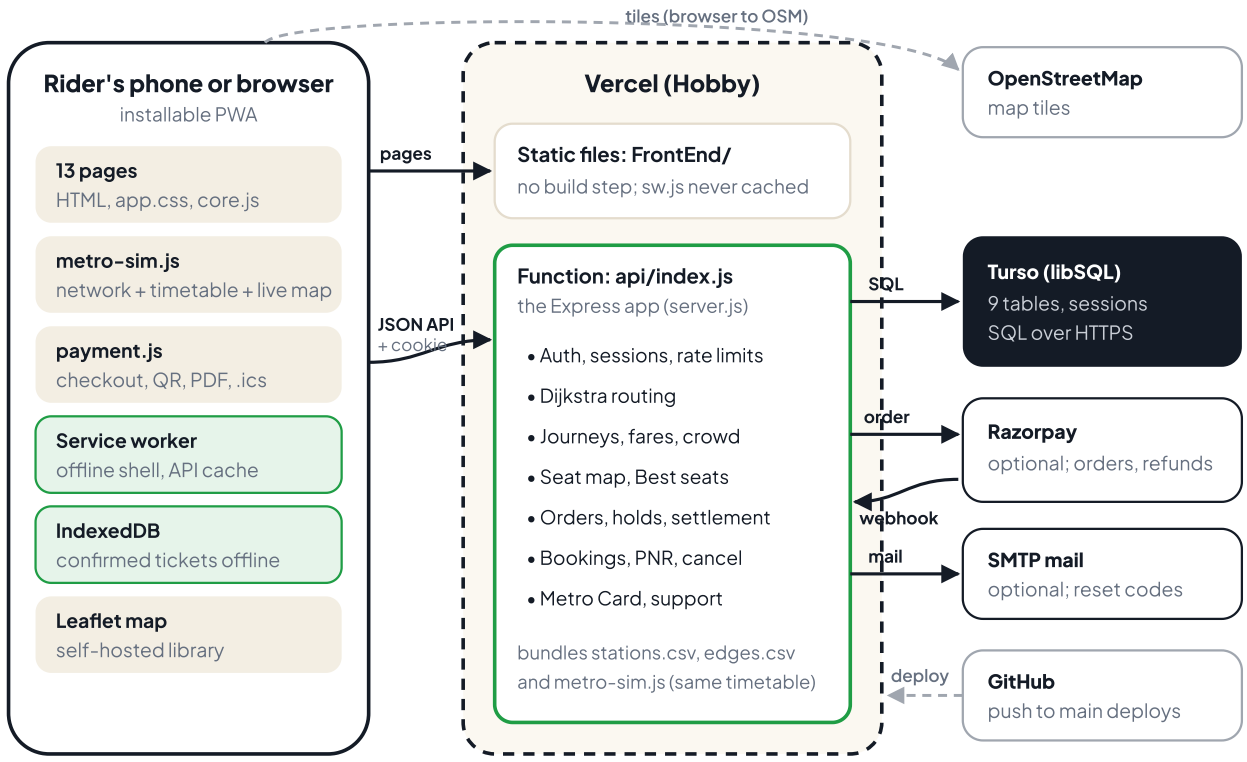


Figure 1. Production architecture. Solid arrows are requests while the app runs; dashed grey links are map tiles and deployment. Locally, `node Backend/server.js` serves the same pages and API on one port with a SQLite file instead of Turso.

The life of one booking

1. **Plan.** The browser posts `/journey` with two stations, a date (up to 30 days ahead) and a time window. The server finds the route with Dijkstra, lists up to 12 departures from the shared timetable, prices each by its time slot, and estimates crowding from the seat occupancy of that exact train.
2. **Choose seats.** `/seat-map` returns the taken seats: simulated other passengers, confirmed bookings, and seats other riders are paying for right now. "Best seats" calls `/suggest-seats`.
3. **Order.** `/payments/order` checks 1–6 valid seat IDs, computes the amount on the server, creates a gateway order, and stores a pending order that holds the seats for 15 minutes. The availability re-check, the release of the rider's older hold and the insert commit as one transaction.
4. **Pay.** The checkout sheet offers UPI, card, net banking or the Metro Card at 10% off. With Razorpay keys, the first three open Razorpay Checkout; in test mode the payment is simulated and no card or UPI details are collected.
5. **Settle.** `/payments/verify` checks the Razorpay HMAC signature, then claims the order (pending → settling) atomically, re-checks the seats, debits the Metro Card if used, issues a PNR, and inserts the booking and one row per seat. The result is stored on the order, so a repeat call or the webhook gets the same answer.

6. **Ticket.** The browser shows the ticket, saves it in IndexedDB and draws the QR code on the device. The rider can save a PDF, share it or add it to the calendar.

7. **Cancel (optional).** Before departure, `/cancel-booking` marks it cancelled, frees the seats and credits the fare to the Metro Card in one transaction.

Key design decisions

DECISION	WHY
One timetable file for server and browser	`metro-sim.js` is a UMD module: `require`d by Node, a `<script>` in the page. The departure a rider books is, by construction, the train on the live map.
The whole Express app as one serverless function	Free hosting with no server to manage, and the same code runs locally with `npm start`.
libSQL: a local file or Turso	The same SQLite SQL everywhere. Turso is reached with the pure-JavaScript web client, so Vercel needs no native module.
Sessions stored in the database	Serverless instances come and go; sign-ins must survive cold starts and restarts.
Transactions found through `AsyncLocalStorage`	Any query inside `transaction(fn)` joins it without passing a handle around; a nested transaction becomes a savepoint.
Seat holds stored on the payment order	Hold, price and booking draft live in one row. Expiry is a timestamp, not a timer, because serverless functions do not keep timers running.
Unique index on (date, train, seat)	The database itself refuses a second sale of the same seat, even if two settlements race past every check above it.
Simulated passengers seeded by SHA-1	Occupancy is stable per train and seat, so every request and every rider sees the same "other passengers" with nothing stored.
No framework, no build step, self-hosted assets	The whole app shell can be cached for offline use, and no third party sees a visitor except map tiles and Razorpay.

API endpoints

METHOD	PATH	PURPOSE
POST	`/register`	Create an account; needs `acceptTerms` and `ageConfirmed`
POST	`/login` · GET `/logout`	Sign in (rate-limited) and out
POST	`/forgot-password` · `/reset-password`	Email an 8-character reset code; use it once to set a new password
POST	`/delete-account`	Password plus typing DELETE; cancels upcoming trips and removes personal data
GET · POST	`/profile` · `/update-profile`	Profile, Metro Card balance, upcoming trips; change name, password, saved stations
GET	`/stations`	Every station node with its line
POST	`/shortest-path`	Route, stops, distance and base fare (Dijkstra)
POST	`/journey`	Route plus up to 12 priced departures with crowd levels for a date and window

METHOD	PATH	PURPOSE
POST	`/seat-map` · `/suggest-seats`	Taken seats for one departure; best group for a party of 1–6
GET	`/payments/config`	Provider (test or Razorpay), public key, Metro Card discount
POST	`/payments/order` · `/verify` · `/cancel`	Create a server-priced order and hold seats; confirm payment; release the hold
POST	`/payments/webhook`	Razorpay events, verified with the webhook secret
GET	`/my-bookings` · `/check-pnr`	Rider's trips; public PNR status without email or payment details
POST	`/cancel-booking`	Cancel before departure; refund to the Metro Card
GET · POST	`/card-balance` · `/support`	Balance and last 30 transactions; send a support message

Next station **05 · The metro network data**

05

CHAPTER 05 · VIOLET LINE

The metro network data

Everything in MetriGo, from routes to fares to the live map, starts from one model of the network: stations as nodes, track sections as weighted edges, and lines as ordered lists of coordinates.



Figure 2. The modelled network, drawn by `documentation/build.js` from the coordinates in `metro-sim.js`. White rings mark interchanges. Red · Yellow · Blue · Green · Violet · Pink · Magenta · Grey · Airport Express

Three files, one network

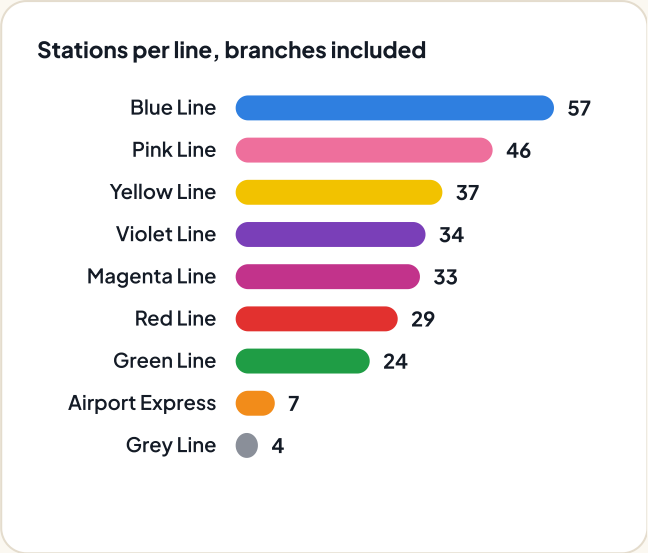
FILE	SHAPE	HOLDS
`Backend/data/stations.csv`	275 rows	`ID, Name, Line`: one node per station per line, so an interchange such as Rajiv Chowk appears once for Yellow and once for Blue
`Backend/data/edges.csv`	293 rows	`From, To, Distance` in km. 31 edges are 0 km: they join the nodes of one interchange
`LINES` in `metro-sim.js`	13 segments	Line code, colour, ordered stations with latitude and longitude; branches share a `group` with their main line

Interchanges

29 stations serve more than one line. Kashmere Gate joins three lines (Red, Yellow and Violet), and Majlis Park has three nodes: the Pink Line, its Maujpur extension and the Magenta Line's extension. The router may start or end on any node of a station, and the path collapses repeated names, so a rider sees "Rajiv Chowk" once.

Where the data comes from

Station order, names and coordinates were taken from OpenStreetMap route relations (© OpenStreetMap contributors, ODbL). Rapid Metro Gurugram and Noida's Aqua Line are separate networks and are not included.



Every line segment

LINE	CODE	STATIONS	TERMINI	END TO END
Red Line	RD	29	Rithala – Shaheed Sthal	79 min
Yellow Line	YL	37	Samaypur Badli – Millennium City Centre Gurugram	108 min
Blue Line	BL	50	Dwarka Sector 21 – Noida Electronic City	131 min
Blue Line · Vaishali branch	BB	8	Yamuna Bank – Vaishali	19 min
Green Line	GN	22	Inderlok – Brigadier Hoshiyar Singh	59 min
Green Line · Kirti Nagar branch	GB	3	Kirti Nagar – Ashok Park Main	5 min
Violet Line	VL	34	Kashmere Gate – Raja Nahar Singh	102 min
Pink Line	PK	38	Majlis Park – Shiv Vihar	122 min
Pink Line · Maujpur extension	PB	10	Maujpur-Babarpur – Majlis Park	28 min
Magenta Line	MA	26	Botanical Garden – Krishna Park Extension	83 min
Magenta Line · Majlis Park extension	MX	7	Deepali Chowk – Majlis Park	20 min
Grey Line	GR	4	Dwarka – Dhansa Bus Stand	10 min
Airport Express	AE	7	Yashobhoomi Dwarka Sector 25 – New Delhi	47 min

End-to-end time is the simulated run time: straight-line distance × 1.12 at an average 34 km/h, plus 30 seconds at each station. Codes appear in train IDs such as `YLD-117`.

Next station **06 · The algorithms**

06

CHAPTER 06 · PINK LINE

The algorithms

These are the parts to study. Each one is written by hand, without a library, in `Backend/server.js` or `FrontEnd/assets/js/metro-sim.js`.

6.1 Shortest route: Dijkstra with a binary heap

The graph has V = 275 nodes and E = 293 undirected edges (each stored in both directions in an adjacency list). `dijkstra(src, dest)` keeps a min-heap of `[cost, node]` pairs, implemented as an array with sift-up and sift-down, and stops as soon as a target node is popped.

- **Many sources, many targets.** The heap starts with every node of the origin station at cost 0, and any node of the destination station ends the search. Changing lines at the start or end is free.
- **Transfer penalty.** A 0 km edge (changing lines) costs 5 km in the search: roughly the walk plus the wait of a change at metro speed. The real distance is tracked separately for the fare.
- **Lazy deletion.** A popped entry whose cost is above the best known cost is skipped, so the heap never needs a decrease-key operation.
- **Path.** A `prev` map rebuilds the path; repeated interchange names are collapsed.

> **Complexity:** O((V + E) log V) time and O(V) space. For a network this size, the whole request (HTTP, JSON, routing and fare) took a median of 0.39 ms in the benchmark.

Example: Rithala → IGI Airport, two changes chosen by the router



The first afternoon departure on 25 October 2026, from `/journey`. The timetable adds a 4-minute change at each interchange.

Base fare by distance

DISTANCE	≤ 2 KM	≤ 5 KM	≤ 12 KM	≤ 21 KM	≤ 32 KM	OVER 32 KM
Base fare	₹10	₹20	₹30	₹40	₹50	₹60

6.2 The timetable and the live map

Trains leave each terminal from 06:00 to 23:00: every 3 minutes in the peaks (08:00–11:00 and 17:00–21:00) and every 5 minutes otherwise. That gives 261 departures per direction per day and 6,786 train runs across the 13 segments. Each line's station offsets are precomputed once from coordinates (haversine distance × 1.12 at 34 km/h, plus a 30-second dwell).

- **Journeys.** `legsForPath` splits a route into legs, one per line and direction. `scheduleLegs` boards the first train of each leg at or after the arrival time plus a 4-minute change.

- **Live positions.** For every line, direction and departure, the elapsed time places the train between two stations; it holds still for the dwell, then moves linearly. All 484 trains at 9 AM are computed in 0.08 ms.
- **Train IDs.** Line code, direction (D down, U up) and the departure index: `YLD-117` is the 17th Yellow Line train heading down the list. IDs are stable for a given date, so a booked train can be found again.

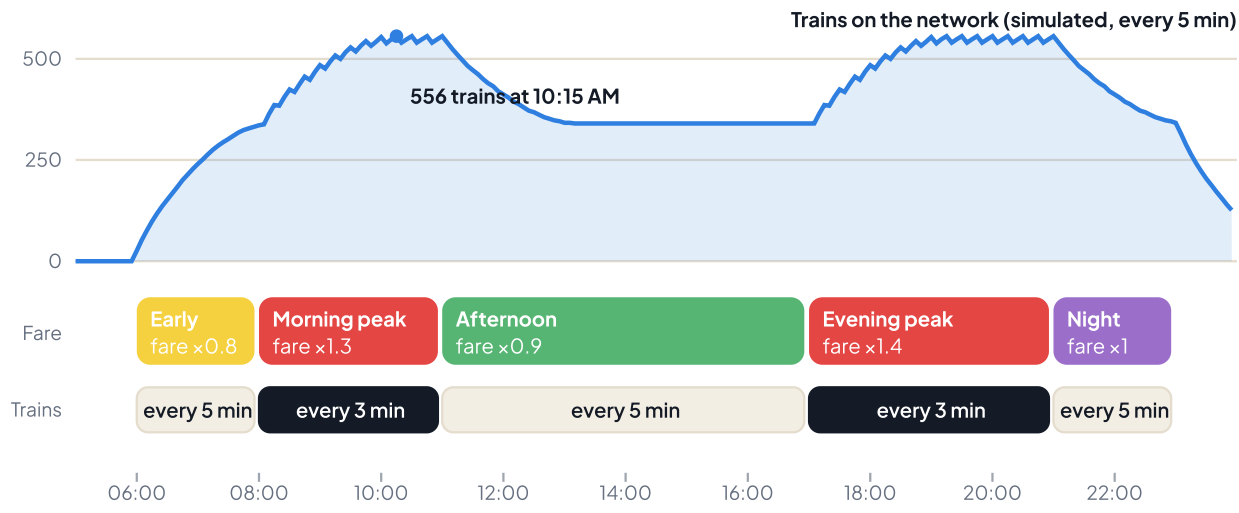


Figure 3. A simulated day: trains on the network (top), fare time slots, and how often trains leave each terminal. All times are Indian Standard Time, whatever the device's time zone.

6.3 Dynamic pricing

TIME SLOT	HOURS	MULTIPLIER	₹30 BASE FARE BECOMES	SEATS TAKEN (SIMULATED)
Early morning	6–8 AM	× 0.8	₹24	12%
Morning peak	8–11 AM	× 1.3	₹39	45%
Afternoon	11 AM–5 PM	× 0.9	₹27	22%
Evening peak	5–9 PM	× 1.4	₹42	50%
Night	9–11 PM	× 1.0	₹30	18%

The slot comes from the departure minute, not the booking time. The Metro Card takes 10% off the order total. The browser shows the same multipliers as a preview, but the server always recomputes the amount from the route and the train.

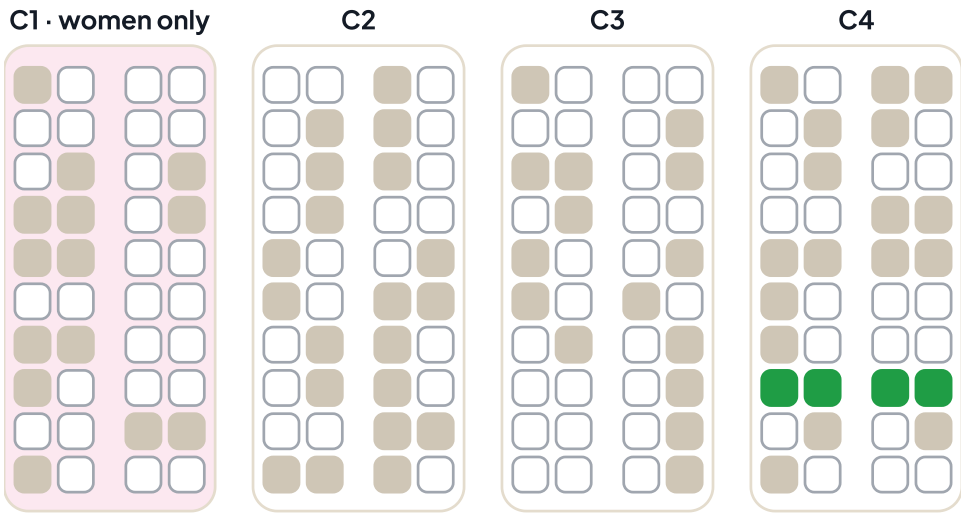
6.4 Seat allocation: greedy, then backtracking

A train has 4 coaches of 10 rows with seats A B | C D, 160 in all. Coach C1 is women-only, as on the Delhi Metro, so suggestions try C2, C3, C4 and only then C1. Simulated passengers fill a share of seats that depends on the slot; a seat is taken when the first byte of SHA-1(date | train | seat) / 256 is below that share. Crowd level is Low below 25% taken, Medium below 45%, otherwise High.

1. **Greedy, O(n) over the free seats.** Walk the coaches in order and each row from the front; return the first run of *k* adjacent free seats. This answers almost every request at once.
2. **Backtracking, only if greedy fails and $k \le 6$.** Within each coach, build groups seat by seat in order. Prune a branch as soon as a seat is more than *k* rows behind the group's first seat. Score each complete group as `rows used × 10 + row spread × 5 + gaps`, and keep the lowest.

3 **Fallback.** For larger groups, take the first *k* free seats.

Train YLD-117, 8:02 AM morning peak: four riders ask for Best seats



Grey seats are taken (66 of 160, crowd "Medium"). Greedy finds no four adjacent seats in C2 or C3 and picks row 8 of C4 (green): C4-8A, C4-8B, C4-8C, C4-8D.

6.5 Hashing

WHERE	HOW
PNR	`"MG"` + 8 digits from SHA-256(email | time | 8 random bytes), regenerated if it already exists
Reset codes	8 random hex characters, stored only as a SHA-256 hash and compared in constant time
Simulated passengers	SHA-1 of date, train and seat: the same seats are "taken" on every request with nothing stored
Lookups	`Map` and `Set` for stations, adjacency and taken seats (O(1) on average); users by primary key, case-insensitive
Passwords	bcrypt with cost 10 (bcryptjs); old scrypt hashes from version 1 still verify

Complexity at a glance

ALGORITHM	TIME	SPACE	WHERE
Dijkstra (binary heap)	O((V + E) log V)	O(V)	`dijkstra()`, server.js
Greedy seats	O(n) over free seats	O(n)	`greedyAllocate()`
Backtracking seats	O(C(n, k)) worst case, cut hard by row pruning	O(k)	`backtrackAllocate()`
Departures for a route	O(D · L) for D departures and L legs	O(L)	`departures()`, metro-sim.js
Live positions	O(lines × departures × stations)	O(trains)	`livePositions()`
Hash lookups	O(1) average	O(n)	Maps, Sets, primary keys

Next station **07 · Benchmark and efficiency**

07

CHAPTER 07 · MAGENTA LINE

Benchmark and efficiency

These numbers were measured on 20 October 2026 by `documentation/benchmark.js`, which starts the real Express app against a throwaway SQLite file and calls it over HTTP on the same laptop (13th Gen Intel Core i5-13450HX, Node v24.13.1, Windows 11).

8,505 / 8,505
Station pairs routed (every station to every 7th station); 0 failed

0.39 ms
Median `/shortest-path` request, end to end; p95 0.86 ms

8.1 ms
Median `/journey` across the network: route, 12 departures, fares and crowd levels

1.69 ms
Best seats for four on a busy peak train (greedy finds them)

83.2 ms
Best seats for six on the same train, where backtracking has to run

0.08 ms
Positions of all 484 trains on the live map at 9 AM

Routing

MEASURE	VALUE	NOTES
Pairs tested	8,505	All 244 stations to every 7th station, same-station pairs skipped
Routes found	8,505	The network is fully connected; no pair fails
Request time, median / p95	0.39 / 0.86 ms	Includes HTTP, JSON parsing, Dijkstra and the fare
Longest route found	51 stops	Raja Nahar Singh → Bahadurgarh City, 79.2 km

Base fare of the 8,505 sampled routes

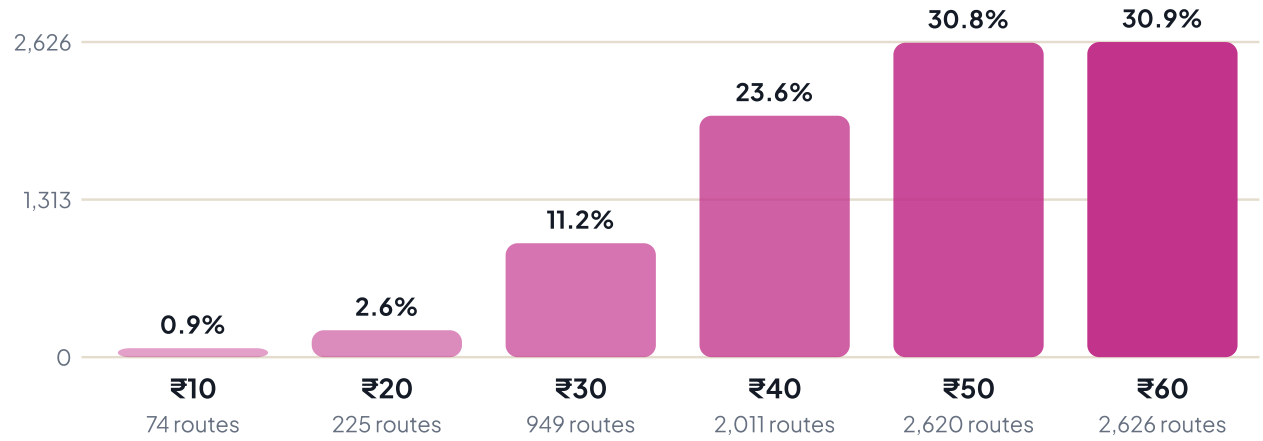


Figure 4. Fares across the network. Most trips are long: 62% cost ₹50 or more, and 31% reach the ₹60 cap (over 32 km).

Seats

PARTY SIZE	SUGGESTED ON YLD-117 (8:02 AM)	MEDIAN REQUEST	PATH TAKEN
1	C2–1A	2.15 ms	Greedy
2	C2–1A, C2–1B	2.01 ms	Greedy
4	C4–8A, C4–8B, C4–8C, C4–8D	1.69 ms	Greedy (C2 and C3 have no free row of four)
6	C3–1B, C3–1C, C3–1D, C3–2A, C3–2B, C3–2C	83.2 ms	Backtracking: six never fit one row of four

In plain words: the router answers any trip on the network in under a millisecond on a laptop and never failed in the test. Planning a full journey with a dozen priced departures takes a few milliseconds. The slowest piece is seating six people together on a crowded train, at about 83.2 ms, because that is the one case where backtracking explores many groups. All of this runs on one CPU core with no cache.

Choices that keep it fast

CHOICE	EFFECT
Graph loaded once at start	CSV parsing and adjacency lists happen once per server instance, not per request
Early exit in Dijkstra	The search stops at the first popped target node instead of settling the whole graph
Precomputed line offsets	Minutes from each terminal to every station are computed once; a departure is then a sum
Greedy before backtracking	The exponential search runs only when the linear pass fails, and only for parties of six or fewer
Hashed occupancy	Simulated passengers cost one SHA-1 per seat and no database rows

These are local numbers. On Vercel, each request also pays network latency, a database round trip to Turso, and a cold start when the function has been idle. Production timings have not been measured.

Next station **08 · Payments and booking integrity**

08

CHAPTER 08 · GREY LINE

Payments and booking integrity

A booking system is judged by what happens when things go wrong at the same moment: two riders on one seat, a double tap on "Pay", a webhook racing the browser. This chapter shows how each case ends.

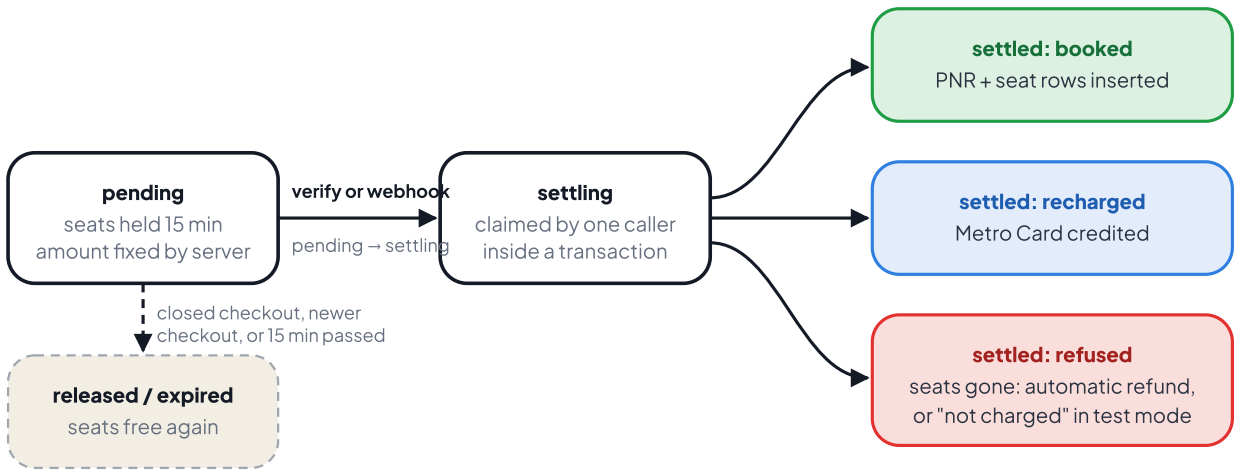


Figure 5. The life cycle of a payment order (`payment_orders.status`). Expired orders are kept 45 more minutes so a late Razorpay payment or webhook can still be booked or refunded, then pruned.

What can go wrong, and what stops it

RISK	PROTECTION
One seat sold twice	Three layers: seats being paid for count as taken; seats are re-checked inside the order and settlement transactions; a unique index on `(travel_date, train_id, seat)` rejects any second insert
One payment settled twice	An in-memory map shares one settlement promise per order within an instance; across instances, `UPDATE … SET status = 'settling' WHERE status = 'pending'` lets only one caller win; the stored result is returned to every later caller; lookups by payment ID catch a payment already turned into a booking
Seats lost after a real payment	The settlement rolls back and the payment is refunded through Razorpay at once. If the refund call fails, the server logs `REFUND NEEDED` with the payment ID and tells the rider to contact support with it
Metro Card overdraft	The debit is `UPDATE … WHERE card_balance >= amount` and the column has `CHECK (card_balance >= 0)`; if it fails, the whole settlement rolls back
Price changed in the browser	The browser sends only stations, date, train and seats. The amount is computed and stored on the server
Forged payment confirmation	HMAC-SHA256 of `order_id|payment_id` with the key secret; webhooks are checked with a separate secret over the raw body; both compared in constant time

RISK	PROTECTION
Abandoned checkout	Closing the sheet calls `/payments/cancel`; a newer checkout replaces the rider's older hold; otherwise the hold simply expires after 15 minutes
Refunding a cancellation twice	The update only matches `status = 'Confirmed'`, and the status change, seat release and credit commit together
The browser never returns after paying	The `payment.captured` webhook settles the order on its own; captured payments for unknown orders are logged for review

Test mode

Without Razorpay keys, `payments.js` creates local order IDs and accepts every payment, and the checkout says so in a banner. It deliberately shows no UPI QR code and asks for no card or UPI details, because a real UPI app could send real money to whoever owns an ID, and a demo should never collect card numbers.

Next station **09 · Database design**

09

CHAPTER 09 · AIRPORT EXPRESS

Database design

SQLite through libSQL: a local file in development and a Turso database on Vercel. The schema is created on start with `CREATE TABLE IF NOT EXISTS`, and columns added later are applied with `ALTER TABLE` when missing.

TABLE	HOLDS	NOTES
`users`	Email (primary key, case-insensitive), password hash, name, saved stations (JSON), Metro Card balance, terms version, consent time, age confirmation	`CHECK (card_balance >= 0)`
`transactions`	Metro Card ledger: recharge, ticket or refund, amount, method, payment ID, PNR, time	Indexed by (email, id); newest first
`bookings`	PNR, rider, status, travel date, train, departure time, plus the full booking as JSON	The API returns exactly what was stored; columns exist for lookups
`booking_seats`	One row per seat of a confirmed booking	`UNIQUE (travel_date, train_id, seat)`; rows deleted on cancel
`payment_orders`	Order ID, rider, purpose, amount, Metro Card amount, draft (JSON), seat-hold keys, expiry, status, result, refund, last error	The seat holds and the settlement state machine
`reset_tokens`	One reset code per account as a SHA-256 hash, expiry, wrong attempts	Burned after 5 wrong tries or one use
`web_sessions`	Session ID, session JSON, expiry	A custom `express-session` store; pruned on about 2% of writes
`support_tickets`	Ticket number, rider, topic, message, time	Topics from a fixed list
`meta`	Key and value	Records that the legacy JSON import ran

How transactions work

`transaction(fn)` opens a libSQL write transaction and runs `fn` inside `AsyncLocalStorage`. Every statement made anywhere inside it, however deep in the call stack, finds that transaction and joins it. A nested `transaction()` becomes a `SAVEPOINT`, so the seat insert inside a settlement can fail and roll back on its own while the outer transaction decides what to do.

Migration from version 1

On first start, `db.js` imports `users.json`, `bookings.json` and `support.json` in one transaction: users with their Metro Card history, bookings with their seats (skipping any seat already sold twice in the old data), and support tickets. A `meta` row makes sure this happens once. `npm run copy-to-turso` copies a local database to Turso and is safe to run more than once. Locally, the SQLite file runs in WAL mode with a 5-second busy timeout, and all database and legacy JSON files are git-ignored.

Next station 10 · Frontend: the web app

10

CHAPTER 10 · RED LINE

Frontend: the web app

Plain HTML, CSS and JavaScript, with no framework and no build step. A soft neumorphic design in light (`#E7E8E4`) and dark (`#2B2C23`), a deep red accent, and the Plus Jakarta Sans typeface. Mobile-first with a bottom tab bar; from 1,024 px wide it becomes a sidebar layout.

PAGE	WHAT IT DOES
Welcome	Sign in, create an account (with consent and age checkboxes) or reset a password; `register.html` forwards here
Home	"Where to?" search, your next trip, saved stations, and a route map with a line filter
Book a trip	Stations, date (next 30 days), part of day and ticket count; then trains with fares and crowd levels; then the seat map and checkout
Trips	Upcoming and past trips, searchable by station or PNR, with the ticket view; works offline
Live Metro	Leaflet map of every simulated train, line filters, follow a train, arrivals board for any station
Metro Card	Balance, recharge (₹50–₹5,000) and transaction history
Profile	Name, password, saved stations, theme, touch sounds, and account deletion
Help	Searchable questions and a support form with topics
Legal	Privacy, terms, refunds and cookies pages, linked from every screen

Ticket states shown in Trips

STATE	MEANING
CONFIRMED	Paid and upcoming. Saved to the phone for offline use; can be cancelled until departure for a full refund to the Metro Card.
COMPLETED	Shown once departure was more than two hours ago. Kept in the trip history; removed from the offline store.
CANCELLED	Cancelled before departure. Seats are released, the fare is credited to the Metro Card, and the PNR still looks up as cancelled.

Shared modules

FILE	LINES	HOLDS
`core.js`	857	Theme, API client, offline ticket store, tab bar, touch sounds and haptics (Web Audio, no sound files), toasts, sheets, station picker with recent stations, route visuals, map helpers, install prompt
`metro-sim.js`	454	Lines and coordinates, timetable, live positions, arrivals, journey legs; also loaded by the server
`payment.js`	451	Checkout sheet with a 15-minute countdown, QR rendering, PDF ticket, share, calendar file

FILE	LINES	HOLDS
`sw.js`	115	Service worker: offline shell and caching rules
`app.css`	948	The design system: colour tokens for both themes, raised and pressed shadows, components

Working offline

REQUEST	STRATEGY
Pages	Stale-while-revalidate: the cached page shows at once and refreshes in the background
CSS, JS, fonts, icons	Cache first, refreshed in the background; the 28-file shell is cached on install
API reads	Network first; offline, the last good response is served with an `X-MetriGo-Offline` header so the page can say so
Money and sign-in	`/payments`, login, logout, reset, cancel, bookings and PNR are never cached
Tickets	Confirmed, upcoming tickets are kept in IndexedDB (without payment details) and open with no signal
Signing out	The service worker deletes the API cache and the ticket database; a new sign-in clears them too

Notable details

- **Boarding-pass PDF.** Drawn with jsPDF at 100 mm wide, with a perforation, the QR code and the route; marked "Demo ticket · Not valid for travel".
- **Calendar file.** An `.ics` event with the route, PNR and seats, and a reminder 15 minutes before departure.
- **Share.** The phone's share sheet sends the PDF itself where supported; otherwise the details are copied.
- **QR codes** are drawn on the device by a vendored library, so they appear underground.
- **Accessibility.** Labelled controls and landmarks (about 80 ARIA attributes across the pages), and reduced motion respected.

Next station **11 · Security, privacy and consent**

11

CHAPTER 11 · YELLOW LINE

Security, privacy and consent

RISK	PROTECTION
Stolen password database	bcrypt with cost 10; older scrypt hashes compared in constant time
Guessing passwords or codes	Per-IP limits per 15 minutes: 10 sign-ins, 5 reset requests, 10 reset attempts, 10 deletion attempts. Behind Vercel, the real client IP is used
Session theft	Cookies are `httpOnly`, `SameSite=Lax` and `Secure` over HTTPS; sessions last 24 hours and are stored server-side
Reset code abuse	8 random hex characters, emailed only and never returned by the API; stored hashed; 15-minute life; 5 wrong tries burn it; used once inside a transaction
Finding out who has an account	"Forgot password" gives the same answer either way and does not wait for the email to send, so timing does not leak it
Two sign-ups at once	The email is the primary key; a racing duplicate is caught as a unique violation
Public PNR lookup	Returns the trip without the rider's email or payment details
Shared phones	Signing out or signing in as someone else clears the cached profile, API responses and saved tickets
Secrets	All keys come from environment variables; `.env` and databases are git-ignored; a missing `SESSION_SECRET` is warned about at start

Privacy and consent

- **Consent is checked on the server**, not only by the form: sign-up fails without `acceptTerms` and `ageConfirmed`, and the user row records the terms version and the time.
- **Account deletion** needs the password and typing DELETE. Upcoming trips are cancelled so their seats free up; Metro Card history, support messages, pending orders, reset codes and sessions are deleted; past bookings are kept without the email, because seat and payment records refer to them.
- **No trackers.** Fonts and libraries are served from the app itself. The only third parties a visitor reaches are OpenStreetMap, when a map is opened, and Razorpay, when paying live.
- **Clear labelling.** Every ticket and PDF says it is a demo and not valid for travel, and the legal pages describe a demo with test-mode payments.

Known gaps

CORS reflects any origin with credentials. `SameSite=Lax` cookies keep cross-site requests signed out, but restricting CORS to the app's own origin would be cleaner. No Content-Security-Policy or frame headers are set, and the minimum password length is 6 characters. All three are listed in chapter 19.

Next station **12 · Testing and quality**

12

CHAPTER 12 · BLUE LINE

Testing and quality

This is the weakest area of the project and is stated plainly: MetriGo has no automated test suite and no CI yet. What is checked today, and how, is below.

CHECK	HOW	AUTOMATED
Every station reachable	`benchmark.js` routes 8,505 pairs through the real API; all succeed	SCRIPT
Journeys, seats, fares by slot	`benchmark.js` calls `/journey`, `/seat-map` and `/suggest-seats` for every slot and party size	SCRIPT
Booking and payment flows	Manual runs in test mode: book, pay by each method, cancel, refund to Metro Card, recharge	MANUAL
Offline tickets	Manual: install the app, go offline, open saved tickets and the QR	MANUAL
Layouts and themes	Manual on phone and desktop widths, light and dark	MANUAL
Concurrency	Enforced by design (transactions, atomic claim, unique index) but not yet tested under parallel load	NOT TESTED

The test plan to add next

1. **Unit tests** with Node's built-in `node:test` (no new dependency): Dijkstra on small graphs and the transfer penalty, fare slabs and multipliers, greedy and backtracking seat groups, PNR format, timetable offsets and train IDs.
2. **Integration tests** on a temporary SQLite file, like the benchmark already does: register, order, verify, cancel; two settlements of one order; two orders for one seat; Metro Card overdraft; webhook before verify.
3. **Browser tests** with Playwright: the booking flow on phone and desktop widths, and tickets opening offline.
4. **CI** with GitHub Actions running all three on every push, so Vercel deploys only tested code.

Next station 13 · Deployment and operations

13

CHAPTER 13 · GREEN LINE

Deployment and operations

PART	WHERE	HOW IT DEPLOYS
Pages	Vercel static files	`outputDirectory: FrontEnd`, no build command; `sw.js` served with `Cache-Control: no-cache`
API	Vercel serverless function	`api/index.js` exports the Express app; rewrites send the API paths to it; bundles the CSVs and `metro-sim.js`; 30-second limit
Dependencies	Root `package.json`	Mirrors `Backend/package.json` so Vercel installs the function's modules
Database	Turso (hosted libSQL)	Schema created on first request; data copied with `npm run copy-to-turso`
Source	GitHub	Every push to `main` redeploys

Configuration

VARIABLE	EFFECT
`PORT`	Local port; default 8080
`SESSION_SECRET`	Signs session cookies. Without it a random one is used and everyone is signed out on restart
`TURSO_DATABASE_URL`, `TURSO_AUTH_TOKEN`	Use Turso (needed on Vercel); otherwise a local SQLite file
`METRIGO_DB`	Path of the local database; default `Backend/data/metrigo.db`
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	Take real payments through Razorpay Checkout instead of test mode
`RAZORPAY_WEBHOOK_SECRET`	Verify webhooks sent to `/payments/webhook`
`SMTP_HOST`, `_PORT`, `_USER`, `_PASS`, `MAIL_FROM`	Email reset codes; without SMTP they are printed in the server console

Running it locally. With Node.js 22 or newer: `cd Backend`, `npm install`, `npm start`, then open `http://localhost:8080`. The server serves the pages too, so nothing else runs. A phone on the same Wi-Fi can open the computer's IP address on port 8080.

Operational rules

- **Change** `TERMS_VERSION` in `server.js` whenever the terms or privacy policy change, so new sign-ups record the version they accepted.
- **Bump** `VERSION` in `sw.js` whenever the list of shell files changes; old caches are deleted on activation.
- **Set** `SESSION_SECRET` in production, or every cold start can sign riders out.
- **Rewrite the legal pages first**, with the operator's details, a grievance officer and real refund terms, before setting Razorpay keys.

Next station **14 · Performance and cost**

14

CHAPTER 14 · VIOLET LINE

Performance and cost

Time per request (laptop, one process, median)

REQUEST	MEDIAN	WHAT IT DOES
`/shortest-path`	0.39 ms	Dijkstra and the fare
`/journey`	8.1 ms	Route, 12 departures, prices, 12 crowd estimates (each reads the database)
`/suggest-seats`, 4 riders	1.69 ms	Occupancy, holds and the greedy pass
`/suggest-seats`, 6 riders	83.2 ms	Backtracking on a crowded train
Live map, all trains	0.08 ms	In the browser; repeated as the map animates

Production adds the round trip to Vercel and to Turso for each database query, plus a cold start when the function has been idle. Pages themselves come from the service worker's cache after the first visit, so switching tabs never waits on the network.

Running cost

SERVICE	PLAN	COST	LIMIT THAT MATTERS
Vercel	Hobby	$0	Personal, non-commercial use; function time limits
Turso	Free	$0	Storage and monthly read/write allowances
GitHub	Free	$0	Public repository
OpenStreetMap tiles	Public tile server	$0	Tile usage policy; heavy traffic needs another tile provider
Razorpay	Test mode	$0	Fees apply only to live payments
SMTP	Optional	$0	Codes print to the console when not set
Total		**$0 / month**	Check each provider's current limits before launch

Next station **15 · How MetriGo compares with other tools**

15

CHAPTER 15 · PINK LINE

How MetriGo compares with other tools

This comparison is about approach and features, based on how these kinds of tools generally work. MetriGo is a demo: it sells no valid tickets and uses no live data, so it does not compete with an operator's official app. Check each product's current features before citing one.

CAPABILITY	METRIGO	OFFICIAL OPERATOR APP	MAP-BASED JOURNEY PLANNER
Shortest route across all lines	YES Dijkstra, explained	Yes	Yes, often multimodal
Valid tickets	NO demo QR only	Yes	Rarely
Live train positions	SIMULATED from a timetable	Varies	Varies by city
Seat selection	EXERCISE metro seats are unreserved	No	No
Fare by time of day	MODELLED	Per operator rules	Shows operator fares
Tickets offline	YES IndexedDB + service worker	Varies	Not applicable
Open, readable source	YES	No	No
No trackers	YES self-hosted assets	Varies	Usually not
Scale and support	Free tier, one person	Operator-backed	Commercial

Advantages, in plain words

1. **The algorithms are visible.** Every route, departure, fare and seat suggestion comes from code a student can read and explain, with its complexity stated.
2. **Integrity is designed in.** Seat holds, atomic settlement, a unique index and automatic refunds handle the cases that break naive booking systems.
3. **Works in a tunnel.** Tickets and their QR codes open with no signal.
4. **Private by default.** No trackers, self-hosted fonts and libraries, consent recorded, and self-serve deletion.
5. **Light and cheap.** No framework, no build step, one function, and $0 a month.

Next station 16 · Limitations

16

CHAPTER 16 · MAGENTA LINE

Limitations

Read this chapter before relying on MetriGo or describing it to others.

Data and simulation

- Station order and coordinates come from OpenStreetMap and may lag behind new openings. Rapid Metro Gurugram and the Aqua Line are not included.
- Distances are straight-line × 1.15, not track lengths, so fares near a slab boundary can differ from reality.
- Every line runs at the same average speed (34 km/h), so Airport Express trips look slower than they are, and it uses normal fares.
- Timetables and train positions are simulated; there are no live feeds, delays or disruptions.
- "Other passengers" and crowd levels are simulated from hashes, not real demand.

Product

- Not affiliated with DMRC. Tickets do not open gates, and real Delhi Metro seats cannot be reserved.
- Payments are simulated unless Razorpay keys are set; going live also needs rewritten legal pages.
- Up to 6 seats per booking, travel up to 30 days ahead, recharges of ₹50–₹5,000.
- Booking, cancelling and recharging need a connection; only viewing tickets works offline.

Engineering

- No automated tests or CI (chapter 12).
- The in-memory settlement map works within one serverless instance; across instances the database claim is what protects, so a second caller may briefly see "still being processed".
- Rate limits are kept in memory per instance, so on serverless they are approximate.
- CORS is open with credentials, no security headers are set, and passwords need only 6 characters.
- Production timings have not been measured; the benchmark is local.
- `FrontEnd/script.js` and `style.css` from version 1 are still in the repository but no page uses them.

Next station **17 · Version 1 versus version 2**

17

CHAPTER 17 · GREY LINE

Version 1 versus version 2

The features promised in version 1's `MetriGo_README.txt`, and where each stands now.

VERSION 1 ITEM	STATUS	NOTES
User authentication	DONE	Plus database sessions, reset codes by email, rate limits and account deletion
Booking with PNR generation	DONE	SHA-256 PNRs checked for uniqueness; public PNR lookup
Shortest path (Dijkstra)	DONE	Rewritten in JavaScript with a binary heap and a transfer penalty; the C module named in the README is not part of version 2
Multi-seat selection	DONE	1–6 seats over 4 coaches with a women-only coach
Smart seat suggestion	DONE	Greedy, then pruned backtracking
Dynamic pricing and time slots	DONE	Five slots from ×0.8 to ×1.4, priced by the train's departure time
Crowd level simulation	DONE	Seeded occupancy per slot plus real bookings and holds
Fast user lookup by hashing	CHANGED	The in-memory user map became a database primary-key lookup
JSON file storage	REPLACED	SQLite / Turso with transactions; old JSON imported once
Not in version 1	ADDED	All nine lines; shared timetable and live map; payments, holds and refunds; Metro Card; PWA and offline tickets; PDF, share and calendar; legal pages and consent; Vercel deployment

Housekeeping: `MetriGo_README.txt` still describes JSON storage and a C module. `README.md` is current; the text file should be updated or marked as the version 1 write-up.

Next station **18 · Complete inventory of everything used**

18

CHAPTER 18 · AIRPORT EXPRESS

Complete inventory of everything used

Languages and formats: JavaScript (Node.js 22+ on the server, modern browsers on the client) · HTML · CSS · SQL (SQLite dialect) · JSON · CSV · iCalendar (`.ics`) · Web App Manifest.

Server libraries

LIBRARY	VERSION	USED FOR
Express	5.1	HTTP API and static files
express-session	1.18	Sessions, with a custom libSQL store
express-rate-limit	8.7	Per-IP limits on sign-in, reset and deletion
bcryptjs	3.0	Password hashing in pure JavaScript
@libsql/client	0.18	SQLite locally and Turso over HTTP
cors	2.8	Cross-origin settings
Nodemailer	8.0	Emailing reset codes over SMTP
Node built-ins	—	`crypto` (SHA-256, SHA-1, HMAC, random bytes, scrypt), `async_hooks`, `fs`, `fetch`

Browser libraries and APIs

LIBRARY OR API	VERSION	USED FOR
Leaflet	1.9.4	Live map and route maps (vendored)
jsPDF	2.5.1	Boarding-pass PDF (vendored)
qrcodejs	1.0.0	QR codes on the device (vendored)
Plus Jakarta Sans	5.3.0	Typeface, self-hosted via Fontsource
Razorpay Checkout	v1	Live payments only; loaded when needed
Web platform	—	Service Worker, Cache Storage, IndexedDB, Web Audio, Vibration, Web Share, Clipboard, `matchMedia`

Services and tools

Data: OpenStreetMap route relations (ODbL 1.0) for station names, order and coordinates; OpenStreetMap tiles under the tile usage policy, attributed on every map; photos `metro.jpg` and `login.jpg` by the author.

Services: GitHub (code) · Vercel (pages and API) · Turso (database) · Razorpay (optional payments) · any SMTP provider (optional email) · OpenStreetMap (tiles).

Tools: Git · Node.js and npm · Google Chrome (testing, and printing this document) · Windows 11 laptop · Claude Code (AI coding assistant, used to write this document and its benchmark).

Next station **19 · Future work**

19

CHAPTER 19 · RED LINE

Future work

1. **Automated tests and CI** as planned in chapter 12, starting with the algorithms and the settlement races.
2. **Tighter web security:** CORS limited to the app's origin, a Content-Security-Policy and frame headers in `vercel.json`, and a longer minimum password.
3. **Shared rate limits** stored in the database, so they hold across serverless instances.
4. **Real track distances and speeds** per line, so the Airport Express is fast and fares match slabs exactly.
5. **Live data** where an open feed exists (for example GTFS for Delhi), replacing simulated timetables.
6. **Rapid Metro Gurugram and the Aqua Line** as separate networks with their own fares.
7. **Accessibility audit** with a screen reader on the seat map and the live map, the two most visual screens.
8. **Production monitoring:** measure request times on Vercel and Turso and alert on settlement errors such as `REFUND NEEDED`.
9. **Cleanup:** remove version 1's unused `script.js` and `style.css`, and update `MetriGo_README.txt`.

Next station **20 · Glossary for study**

20

CHAPTER 20 · YELLOW LINE

Glossary for study

TERM	MEANING
Graph, node, edge	A set of points (stations) joined by links (track sections). A weighted edge carries a cost, here kilometres.
Dijkstra's algorithm	Finds the cheapest path from a start node by always expanding the closest unfinished node. Correct when no edge cost is negative.
Binary heap, priority queue	A tree stored in an array where each parent is no larger than its children, so the smallest item is found in O(1) and removed in O(log n).
Transfer penalty	An extra cost added for changing lines, so the shortest path is also a sensible one.
Greedy algorithm	Takes the best-looking choice at each step without looking back. Fast, but not always optimal.
Backtracking, pruning	Building candidate solutions step by step and abandoning a branch as soon as it cannot lead to a good answer.
Haversine formula	Distance between two points on a sphere from their latitudes and longitudes.
Headway, dwell, lazy deletion	Time between trains on a line; time a train waits at a station; skipping outdated heap entries when popped instead of updating them.
SHA-256, SHA-1	Hash functions: the same input always gives the same fixed-length output, and the output reveals nothing useful about the input.
HMAC	A hash keyed with a secret; proves a message came from someone who knows the secret and was not changed.
bcrypt	A deliberately slow password hash with a built-in salt, to make guessing expensive.
PNR	Passenger Name Record: the booking reference printed on the ticket.
Transaction, savepoint	A group of database changes that all happen or none do; a savepoint lets part of a transaction roll back alone.
Unique index	A database rule that no two rows share the same values in the given columns.
Idempotent	Safe to repeat: doing it twice has the same effect as once, as with settling a payment.
Webhook	A request a service (here Razorpay) sends to your server when something happens.
AsyncLocalStorage	A Node.js feature that carries a value (here the open transaction) through all asynchronous calls of one task.
Serverless function, cold start	Code the host runs on demand; the delay when an idle function is started again.
PWA, service worker	A web app that installs like a native app; a background script that can answer requests from a cache.
Stale-while-revalidate	Serve the cached copy now and fetch a fresh one for next time.
IndexedDB	A database inside the browser, used here for offline tickets.
CORS, SameSite	Browser rules for which sites may call an API, and when cookies are sent with cross-site requests.
ODbL	Open Database License: OpenStreetMap's licence; requires attribution and sharing derived data alike.

Next station **21 · Licences, credits and publishing notes**

21

CHAPTER 21 · BLUE LINE · TERMINUS

Licences, credits and publishing notes

Licences

- **MetriGo's code** has no licence file yet, which by default means all rights are reserved. Add one (for example MIT) if others should be able to reuse it.
- **Network data** derived from OpenStreetMap (`stations.csv`, `edges.csv`, the `LINES` table) is available under the ODbL 1.0, © OpenStreetMap contributors.
- **Bundled libraries:** Leaflet (BSD 2-Clause), jsPDF (MIT), qrcodejs (MIT), Plus Jakarta Sans (SIL Open Font License 1.1). Licence texts are in `FrontEnd/assets/licenses/` and listed in `THIRD_PARTY_NOTICES.md`.
- **Server dependencies** are installed from npm under their own licences.

Publishing notes

- MetriGo is a student project and is **not affiliated with DMRC**. "Delhi Metro" and line names are used only to describe the network.
- Tickets and PDFs are marked "Demo · Not valid for travel"; keep that wording anywhere the app is shown.
- The privacy policy, terms and refund policy describe a demo. Before taking real payments, rewrite them with the operator's legal details and change `TERMS_VERSION`.
- Vercel's Hobby plan is for personal, non-commercial projects; a commercial launch needs a paid plan.

Credits

- **Lanchenba Mutum:** creator of MetriGo. Design, development, algorithms, data preparation and deployment.
- **OpenStreetMap contributors:** the station data and map tiles.
- The authors of Express, libSQL, Leaflet, jsPDF, qrcodejs, bcryptjs, Nodemailer and Plus Jakarta Sans.